Capstone Project: Make the Computer Draw a Box

Nand to Tetris - I (ETCCNT301) | School of Engineering & Technology, K.R. Mangalam University

Name	Lakshay	**Roll No**	2501730367
My K	**K = 67** (last two digits of the roll number)	**K mod 3 / K mod 4**	6 / 7
Box	starts at row 0, 20 rows tall, 16 x ((0 mod 4) + 1) = **16 pixels wide** (1 word) from the left edge	**Extra feature**	K mod 3 = 1: labels and variables saved in symbols.txt

Part A - Building the computer

I joined the chips into **CPU.hdl**, **Memory.hdl** and **Computer.hdl**. In the CPU, bit 15 of the instruction decides between an A-instruction and a C-instruction. The A register loads the instruction itself for an A-instruction, or the ALU output when the dest A bit is set. The second ALU input is A or M, chosen by the a-bit (bit 12), and the six control bits (11 to 6) go straight to the ALU. The D register loads only for a C-instruction with the D dest bit, and *writeM* is bit 15 AND bit 3. For jumps I made *positive* = NOT zr AND NOT ng, combined it with the three jump bits, and AND-ed the result with the C-bit before it goes to the load pin of the PC (inc is always 1).

Memory uses address bit 14 to choose between RAM16K (0 to 16383) and the upper half; in the upper half bit 13 chooses Screen (16384 to 24575) or the Keyboard (24576). Computer is just ROM32K, CPU and Memory wired in a loop.

Built-in chips: inside these three files the ALU, PC, ARegister, DRegister, RAM16K, Screen, Keyboard and ROM32K are used by name, and the built-in versions of these chips were used for the test runs below.

Test file	Chip	Result
CPU.tst	CPU.hdl	passed - no differences with CPU.cmp
Memory.tst	Memory.hdl	passed - no differences with Memory.cmp
ComputerAdd.tst	Computer.hdl	passed (Add.hack, R0 = 5)
ComputerMax.tst	Computer.hdl	passed (Max.hack)
ComputerRect.tst	Computer.hdl	passed (Rect.hack)

Hardware Simulator - CPU.tst

time	inM	instruction	reset	outM	writeM	addre	pc	DRegiste
0+	0	0011000000111001	0	0	0	0	0	0
1	0	0011000000111001	0	0	0	12345	1	0
1+	0	1110110000010000	0	12345	0	12345	1	12345
2	0	1110110000010000	0	12345	0	12345	2	12345
2+	0	0101101110100000	0	0	0	12345	2	12345
3	0	0101101110100000	0	0	0	23456	3	12345
3+	0	1110000111010000	0	11111	0	23456	3	11111
4	0	1110000111010000	0	12345	0	23456	4	11111
4+	0	0000001111101000	0	-11111	0	23456	4	11111
5	0	0000001111101000	0	-11111	0	1000	5	11111
5+	0	1110001100001000	0	11111	1	1000	5	11111
6	0	1110001100001000	0	11111	1	1000	6	11111
6+	0	0000001111101001	0	-11111	0	1000	6	11111
7	0	0000001111101001	0	-11111	0	1001	7	11111
7+	0	1110001110011000	0	11110	1	1001	7	11110
8	0	1110001110011000	0	11109	1	1001	8	11110

...

CPU.tst | output compared with CPU.cmp | no differences

Fig. 1 - CPU.tst result (first lines of the output)

Hardware Simulator - Memory.tst

in	load	address	out
12345	1	0100000000000000	0
12345	1	0100000000000000	12345
12345	1	1000000000000000	0
12345	1	1000000000000000	12345
-1	1	0000000000000000	0
-1	1	0000000000000000	-1
9999	0	0000000000000000	-1
9999	0	0000000000000000	-1
9999	0	0100000000000000	12345
9999	0	1000000000000000	12345
12345	1	0000000000000000	-1
12345	1	0000000000000000	12345
12345	1	1000000000000000	12345
12345	1	1000000000000000	12345

...

Memory.tst | output compared with Memory.cmp | no differences

Fig. 2 - Memory.tst result (K and Y keys held when the test asks for them)

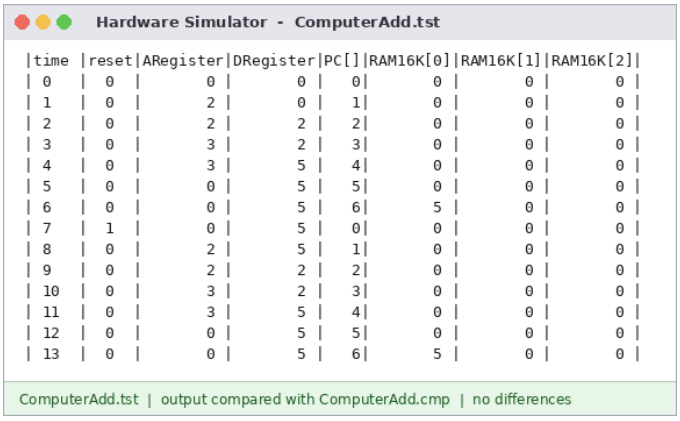


Fig. 3 - ComputerAdd.tst result

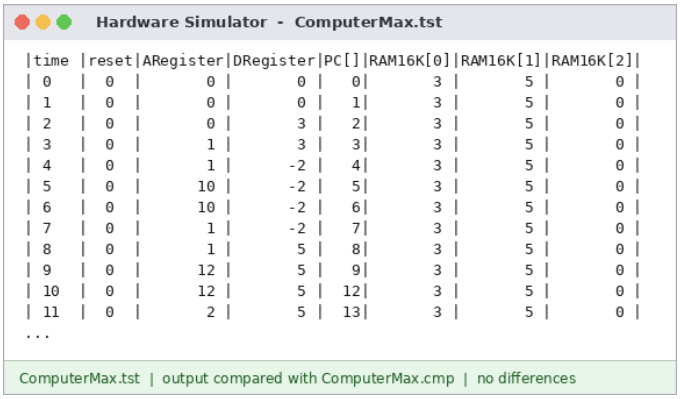


Fig. 4 - ComputerMax.tst result

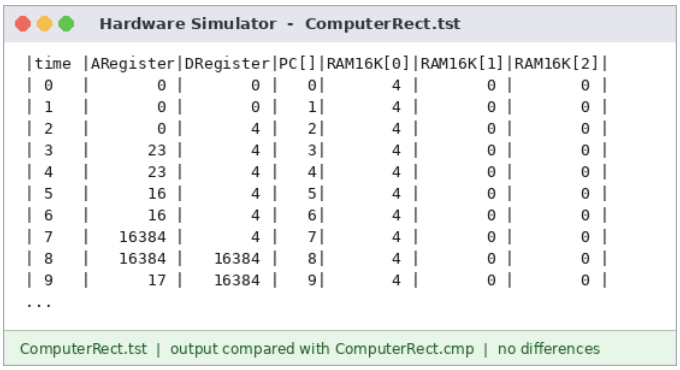


Fig. 5 - ComputerRect.tst result

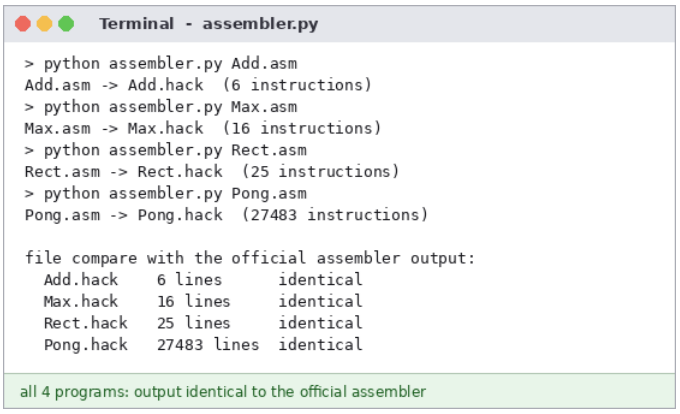


Fig. 6 - assembler.py output compared with the official assembler

Part B - The assembler (assembler.py)

The assembler is written in Python and reads the program twice. In the **first reading** I remove comments, spaces and blank lines, and every label like (LOOP) is stored with the address of the next real instruction (a label line itself produces no code). In the **second reading** every line becomes 16 bits. An A-instruction is 0 followed by the 15-bit value; if the name is not a number it is looked up in the predefined symbols (R0 to R15, SP, LCL, ARG, THIS, THAT, SCREEN, KBD), then in the labels, and otherwise it is a new variable which gets RAM address 16, 17, 18 and so on. A C-instruction is 111 + comp (a-bit and six bits) + dest (three bits) + jump (three bits), taken from small tables.

I tested it on Add.asm, Max.asm, Rect.asm and Pong.asm (and the versions without symbols). The binary output is identical to the output of the official assembler for all of them:

Program	Instructions	Same as official .hack file?
Add.asm	6	yes
Max.asm	16	yes
Rect.asm	25	yes
Pong.asm	27483	yes

Extra feature for K = 0 (K mod 3 = 0): after assembling, the program writes **symbols.txt** with every label and variable and its address. For my Box.asm it contains:

```
// labels
START 0
DRAW 8
ROW 16
// variables
color 16
addr 17
count 18
```

Part C - The box program (Box.asm)

For K = 0 the box starts at row 0, is 20 rows tall and is 16 x ((0 mod 4) + 1) = 16 pixels wide, which is exactly **one word** at the left edge. Row r starts at SCREEN + 32*r, so the 20 words to write are 16384, 16416, ... up to 16384 + 32*19 = 16992. I built the program in small steps (one dash, a wider dash, a box with a loop) and then added the key check.

How it works: (1) set the colour variable to 0 (white); (2) read KBD and if it is not 0 set the colour to -1 (all 16 pixels black); (3) put the first address in *addr* and 20 in *count*; (4) loop: write the colour at *addr*, add 32 to *addr*, decrease *count* and repeat while it is above 0; (5) jump back to the start so the keyboard is checked again and again. When the key is released the same loop writes 0, so the box disappears.

I converted it with **my own assembler** (python assembler.py Box.asm), which gave Box.hack with 31 instructions, and ran Box.hack in the CPU Emulator. With a key held, rows 0 to 19 of the first screen word are -1 (all pixels black) and every other screen word stays 0; after releasing the key the whole screen is white again.



Fig. 7 - Box.hack running: key pressed, key released, and a zoom of the box (16 pixels wide, 20 rows tall, top-left corner)

Box.asm (comments about K shortened here, full file is in the capstone folder):

```
(START)
    @color
    M=0             // default colour = white (no key)
    @KBD
    D=M
    @DRAW
    D;JEQ           // KBD = 0 -> no key, keep white
    @color
    M=-1            // key pressed -> black (all 16 bits set)

(DRAW)
    @SCREEN
    D=A
    @addr
    M=D             // addr = first word of row 0
    @20
    D=A
    @count
    M=D             // 20 rows to paint

(ROW)
    @color
    D=M
    @addr
    A=M
    M=D             // paint one word = 16 pixels of this row
    @32
    D=A
    @addr
    M=D+M           // move to the next row (+32 words)
    @count
    MD=M-1
    @ROW
    D;JGT           // more rows left?
    @START
    0;JMP           // check the keyboard again
```

One mistake I fixed

When I first ran my assembler on Rect.asm it crashed with a KeyError for 'DM'. My destination table only had the spelling **MD**, like the table in the book, but Rect.asm has the line *DM=M-1* (and Pong.asm has *DM=M+1*), and the

official assembler accepts both orders. Adding more spellings one by one felt wrong, so I deleted the table and wrote a small function that looks at the dest field and sets the A, D and M bits if that letter is present. Now MD, DM, AM, MA and so on all work. After this fix Rect.asm and Pong.asm matched the official .hack files exactly.

Files in the capstone/ folder

File	What it is
CPU.hdl, Memory.hdl, Computer.hdl	Part A - the computer
assembler.py	Part B - two-pass Hack assembler with the symbols.txt feature
symbols.txt	labels and variables of Box.asm
Box.asm, Box.hack	Part C - the box program and its binary made with assembler.py
Capstone_Report.pdf	this report